Logistic Regression + TF-IDF model 1 of 6

The interpretable baseline. Every word gets a weight; the weights add up to a decision.

Each word carries a learned weight. Positive words push the score up, negative words push it down; the sum is squashed to a probability. You can read every weight, so nothing is hidden.

How it works

TF-IDF turns each review into a long, sparse list of word counts, weighted so common words fade and distinctive words stand out. Logistic regression learns one weight per word, adds them for a given review, and squashes the total into a probability with the logistic (sigmoid) function.

Plain version: It scores every word as a little vote for positive or negative and adds the votes up. Because each word has a visible weight, you can read exactly what it trusts.

Good at

- Fully interpretable: print the top positive and negative words
- Trains in seconds, tiny to store and serve
- A genuinely strong bar to beat

Watch out

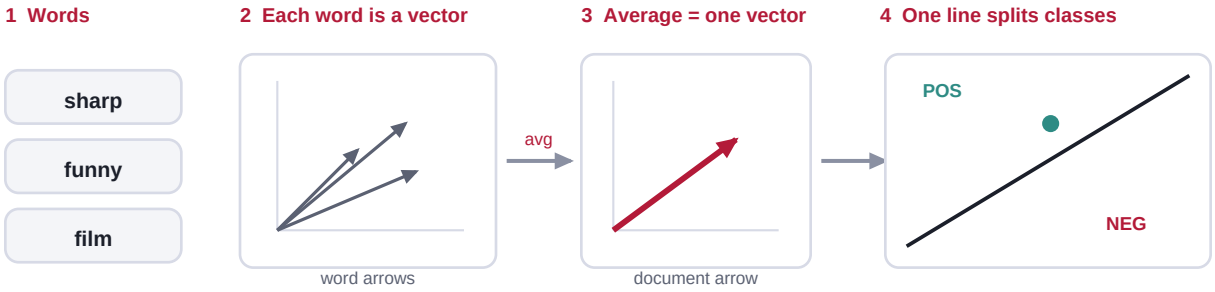
- Bag of words, so it misses word order unless you add n-grams
- Cannot capture deeper meaning or long-range context

At a glance

On the bake-off: about 0.77 accuracy, ~3 s to train, a few MB.

FastText model 2 of 6

A fast linear model with embeddings. Look up, average, one line.



Turn each word into a little arrow, average all the arrows in a review into one arrow, then draw a single line to split positive from negative. Just look-up, average, one line, which is why it trains in seconds.

How it works

Every word becomes a short vector (an embedding). A review becomes a single vector by averaging the vectors of its words and short word pairs, and one linear layer classifies that average. A shortcut called hierarchical softmax keeps training quick, and a character n-gram trick lets it handle typos and unseen words.

Plain version: It turns each word into a little arrow, averages all the arrows in a review into one arrow, and draws a single line to split positive from negative.

Good at

- Rivals deep networks on many text tasks
- Trains in seconds on a plain CPU
- Handles new or misspelled words through subword pieces

Watch out

- Averaging still loses most word order
- Default model is large unless you cap dim and buckets

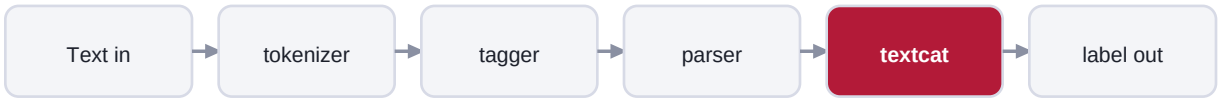
At a glance

On the bake-off: about 0.78 accuracy, ~10 s to train, near 40 MB (with dim=50, bucket=200000).

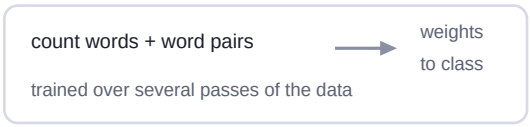
spaCy textcat **model 3 of 6**

A classifier that lives inside a production NLP pipeline.

The spaCy pipeline (an assembly line)



inside textcat (bag of words):



spaCy runs an assembly line of NLP steps. The text categorizer is one station on that line, so the classifier sits right next to your tokenizer, tagger, and entity recognizer in a single production tool.

How it works

spaCy runs an assembly line: tokenizer, tagger, parser, entity recognizer. The text categorizer is one station on that line. In its bag-of-words setting it counts words and word pairs and learns weights, much like the baseline, but it trains through spaCy's own loop over several passes of the data.

Plain version: It is spaCy's own built-in classifier, handy when the model has to sit right next to your other NLP steps in one tool.

Good at

- Drops into an existing spaCy pipeline with no glue code
- Production-focused: fast at prediction time
- Can be upgraded to a neural architecture later

Watch out

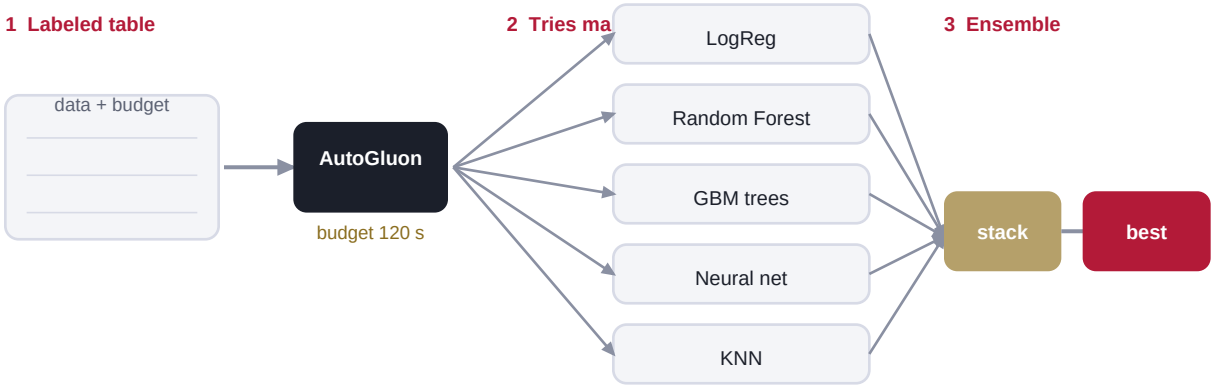
- Slower to train here than the plain linear baseline
- Bag-of-words setting is not more accurate than TF-IDF

At a glance

On the bake-off: about 0.75 accuracy, ~60 s to train, a few MB.

AutoGluon (AutoML) model 4 of 6

A manager, not a model. It builds and blends models for you.



A manager, not a model. Hand it the table and a time budget; it cleans the columns, trains many models, tunes them, and blends the best into an ensemble. You never choose an architecture.

How it works

You hand AutoGluon the labeled table and a time budget. It cleans the columns (turning text into n-gram features), trains many different model types, tunes them, and stacks the best into an ensemble, all automatically. The time budget caps the search so it fits class time.

Plain version: A robot data scientist. Give it the data and two minutes, and it tries a pile of models and hands back the best blend.

Good at

- Strong accuracy with almost no human effort
- Handles mixed data types automatically
- A time budget makes cost predictable

Watch out

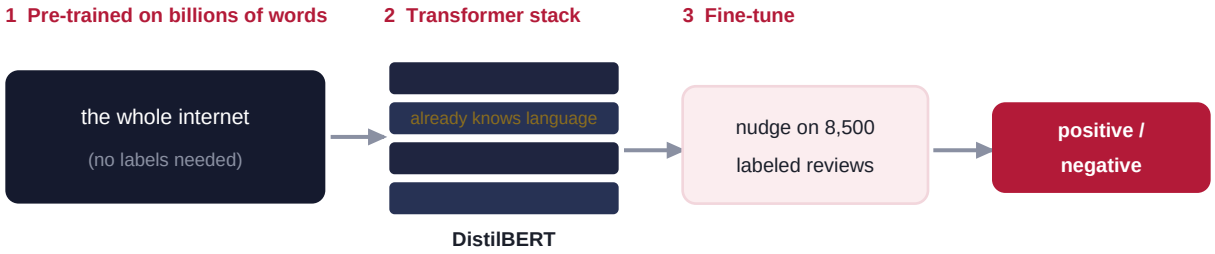
- The ensemble is a black box you cannot easily explain
- Heavy to install and larger to store and serve

At a glance

On the bake-off: about 0.79 accuracy in a 120 s budget, plus a few minutes to install.

DistilBERT (transformer) model 5 of 6

A small transformer that already read the internet, fine-tuned on your labels.



A network that has already read the internet, then gets a light nudge on your few thousand labels.
The most accurate here, and the most expensive: it wants a GPU, is large, and cannot explain itself.

How it works

BERT is a neural network pre-trained on billions of words, so it already carries a rich sense of language; DistilBERT is a compressed, faster copy. Fine-tuning nudges its weights on your few thousand labeled reviews. We build transformers properly in Week 5.

Plain version: A model that has already read the internet and just needs a little pointing at your task. Powerful, heavy, and a black box.

Good at

- The highest accuracy in the lineup
- Captures word order and context, not just word counts
- Transfers knowledge from huge pre-training

Watch out

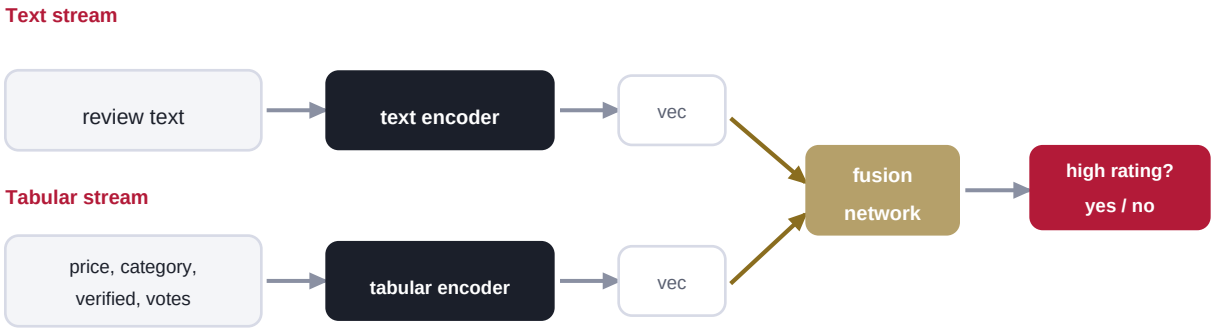
- Wants a GPU and is slow to train
- Hundreds of MB to store
- Cannot tell you why it decided

At a glance

On the bake-off: about 0.84 accuracy, ~2 min on a GPU, near 255 MB.

Multimodal Fusion (AutoGluon) model 6 of 6

Reads the words and the structured columns together, then combines them.



Each column type gets its own encoder: the words go through a language model, the numbers and categories through tabular processing. Their outputs are glued together and a small fusion network is trained on top. Fusion beats either channel alone when text and columns carry partly independent signal.

How it works

AutoGluon's MultiModalPredictor gives each column type its own encoder: the text goes through a pretrained language model that produces an embedding, the numeric and categorical columns get tabular processing, and a small fusion network concatenates the two and is trained end to end to predict the label.

Plain version: It reads the words with a language model, reads the numbers the normal way, glues the two together, and trains one combiner on top. Nobody picks the model.

Good at

- Beats text alone when the columns carry extra signal
- Uses data a text-only model would throw away
- Fully automatic across modalities

Watch out

- When one modality already dominates, fusion mostly adds cost
- Heaviest to run, and opaque

At a glance

In the demo: text only ~0.78, tabular only ~0.79, fused ~0.84. Fusion wins, but check whether it earned its cost.